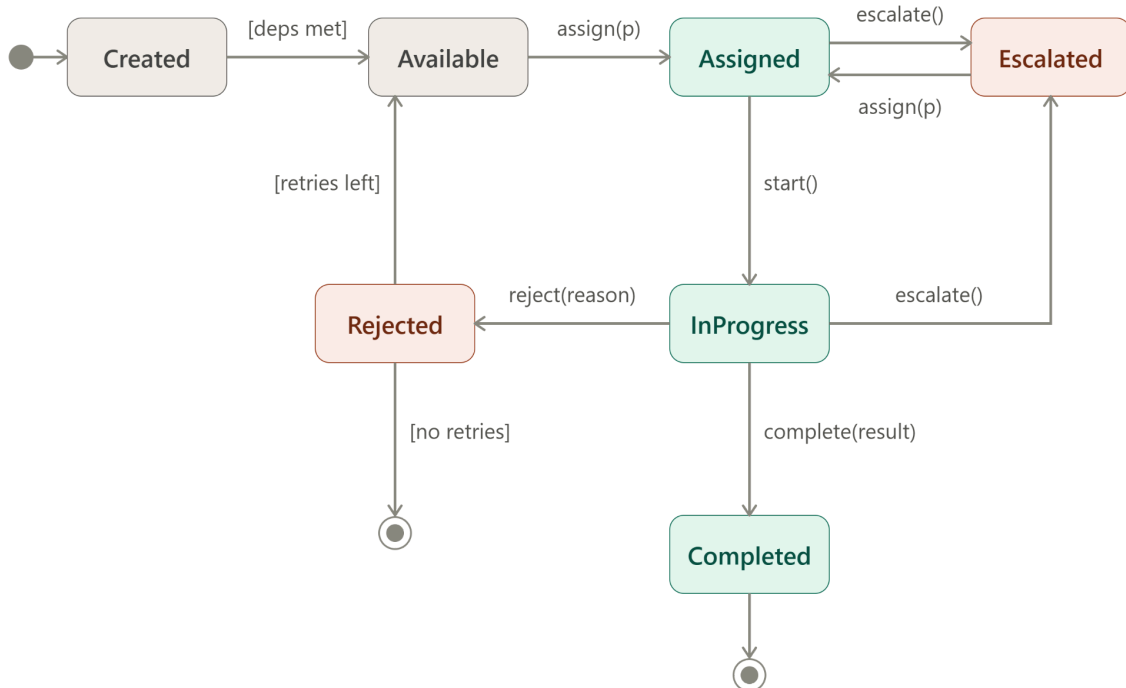


Name: Tshegofatso Kungwane

Student Number: u23605032

Assigned: State Diagram, Sequence Diagram, Factory Method

STATE DIAGRAM

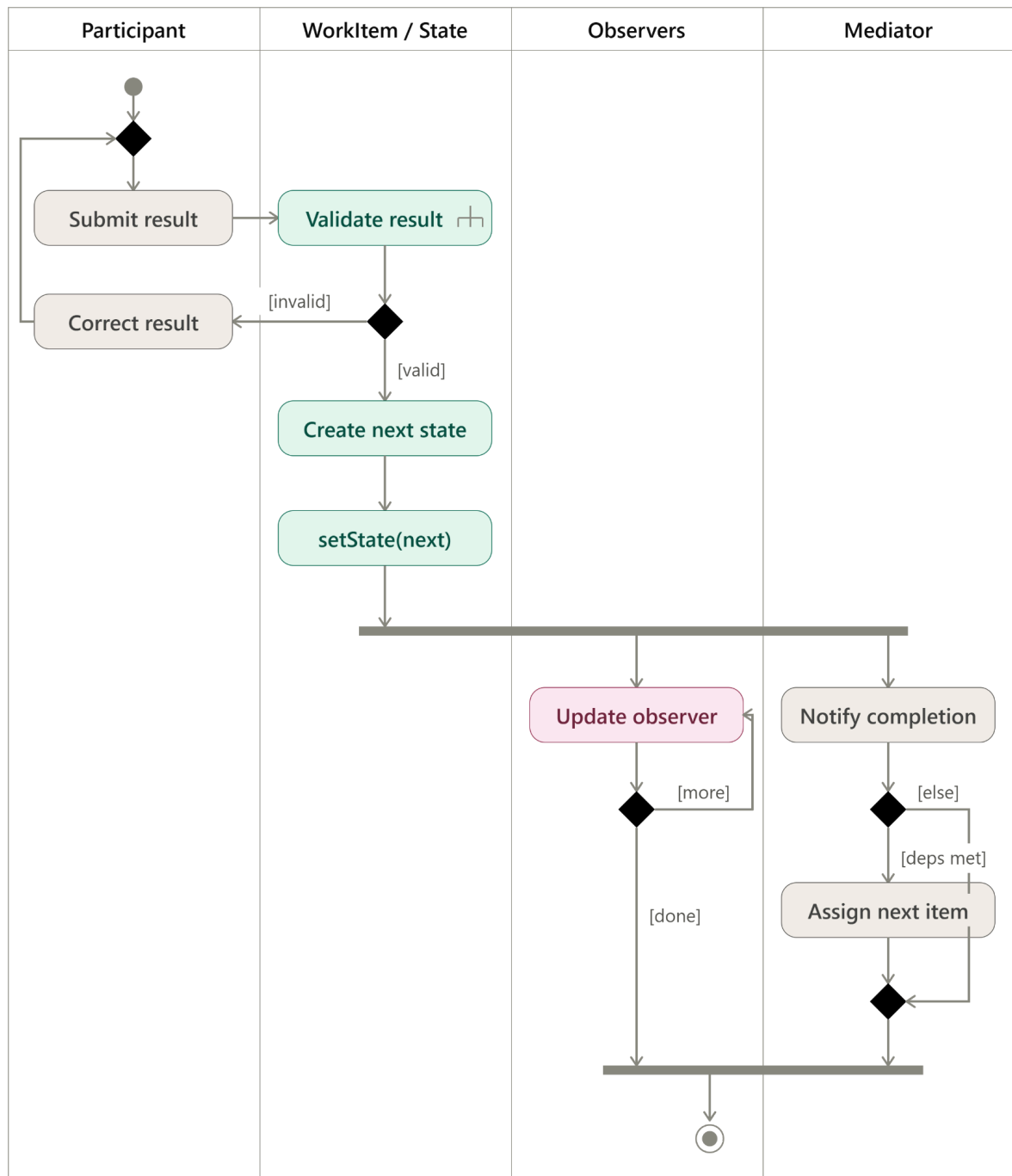


- **Created:** The WorkItem is created and remains in this state until all of its dependencies are completed.
- **Available:** Once all dependencies are completed, the WorkItem becomes available to be assigned.
- **Assigned:** A person is assigned to the WorkItem using `assign(p)`.
- **InProgress:** The assigned person starts working on the WorkItem using `start()`.
- **Completed:** The WorkItem is successfully finished using `complete(result)`. This is a final state.

Handling failures:

- **Rejected:** The WorkItem is rejected using `reject(reason)`.
 - If retries are available, it returns to **Available**
 - If no retries remain, it stays **Rejected** as a final state
- **Escalated:** If the WorkItem is stuck or overdue, `escalate()` moves it to **Escalated**
 - A new `assign(p)` can then move it back to **Assigned**, for example by assigning it to a manager.

ACTIVITY DIAGRAM



Participant Lane

- The flow starts when the Participant submits a result using `complete(result)`
- If the result is **invalid**, the Participant corrects it and submits it again
- This creates a loop back through the merge diamond at the top of the diagram

WorkItem / State Lane

- **Validate result:** The current state checks whether the submitted result is valid
- **Decision:**
 - [invalid] → returns to the Participant so the result can be corrected
 - [valid] → continues with the workflow.
- **Create next state:** The current state creates the next state object
- **setState(next):** The WorkItem changes to the new state
- **Fork:** The flow splits into two parallel branches that run at the same time

Observers Lane

- **Update observer:** Each attached observer is notified using `update(item, event)`
- **Decision:**
 - [more] → loops back and updates the next observer
 - [done] → exits the loop once all observers have been notified

Mediator Lane

- **Notify completion:** The WorkItem informs the mediator that it has been completed using `notifyCompleted(item)`
- **Decision:**
 - [deps met] → the successor's dependencies are satisfied, so the mediator assigns the next item using `requestAssignment(next)`
 - [else] → no assignment is made and the workflow continues
- **Merge:** The different mediator paths rejoin before continuing

End

- **Join:** The bottom join waits for both the Observer and Mediator branches to finish
- The final node ends the workflow

TASK 4: FACTORY METHOD

- TIVIDY has different types of work items: tasks, approvals and subprocesses
- Each type must be created differently, but if/else and switch statements are not allowed (Rule 6)
- Each workflow instance needs its own copy of the data

Participants

- Creator: `WorkItemCreator` - the abstract creator
- ConcreteCreators: `TaskItemCreator`, `ApprovalItemCreator`, `SubprocessItemCreator`
- Product: `WorkItem` - the abstract product
- ConcreteProducts: `TaskItem`, `ApprovalItem`, `SubprocessItem`.

Collaboration

- The system reads a step and finds the correct creator from a registry
- It calls `creator.instantiate(step)`
- The creator creates the correct type of `WorkItem`
- The `WorkItem` is given its data, starts in `CreatedState`, and gets an `AuditObserver`
- The client then uses the `WorkItem` through the `WorkItem` interface

What Varies

- The type of `WorkItem` being created: task, approval or subprocess

Why Factory Method Is Appropriate

- Each type of `WorkItem` has its own creator
- A new type can be added by creating a new product and creator
- No changes to the workflow engine are needed
- The common setup is kept in one place, so every `WorkItem` is created with the correct state and observer

Connection to the Other Diagrams

- State diagram: the factory creates the `WorkItem` in the `Created` state
- State pattern: controls how the `WorkItem` moves between states
- Sequence and activity diagrams: show what happens to the `WorkItem` after it has been created